Infrastructure and process networks are routinely analysed for three separate properties: whether the system keeps functioning when a component fails, how much it can carry, and when a dependent sequence of activities finishes. A water distribution network, a power grid, a plant process line and a construction schedule are frequently the same kind of directed dependency structure examined three different ways, and the three literatures built around these properties, reliability computation, capacity and flow analysis, and project scheduling, developed largely apart from one another, each with its own vocabulary and its own notion of what counts as an exact answer.

All three properties reduce to a computation over the same kind of object, a directed graph whose nodes and edges can fail, vary, or take time to traverse. Reliability computation returns the probability that at least one path still delivers supply from source to demand once every component is subject to failure. Flow analysis returns the greatest throughput the network delivers given the maximum capacity each edge and node sustains, and where it bottlenecks. Scheduling returns the earliest completion time of a dependent sequence of activities and which of them carry no room to slip. Three different quantities, a probability, a capacity and a duration, computed over structures the three literatures largely built their own methods for without citing one another's.

0.1 The Methodological Landscape

A reliability block diagram arranges components in series, parallel, and k-out-of-n groups whose system reliability has a closed form [1], and fault and event trees widen the reachable logic by building the failure mechanism explicitly as a tree of gates [2], both efficient exactly as far as the failure logic reduces to a tree or a series-parallel structure. Path and cut-set decomposition reaches arbitrary topologies exactly by splitting the network into subproblems [3, 4], at a combinatorial cost that grows with the same redundancy that makes the analysis worth doing, and Monte Carlo simulation escapes structure entirely in exchange for exactness, at a sample cost that grows demanding when failures are rare [5]. Binary decision diagrams compile the same Boolean structure into a canonical form on which reliability is a single traversal [6, 7], and cutset conditioning and the junction-tree algorithm reach the same computation from probabilistic inference instead, treating reachability as marginal inference on a directed graphical model [8, 9]. Every one of these methods consumes a point-valued component probability.

Series-parallel reduction collapses independent stretches pair by pair [10], factoring conditions on a single component to split the graph into independent cases [11], and polygon-to-chain transformation widens what can be collapsed in one step beyond a plain series or parallel pair [12]. Treewidth-bounded tree decomposition generalises the same idea to graphs no sequence of series-parallel reductions can fully collapse, bounding the cost of exact computation by a structural width of the graph rather than its raw size

[13], and the hash-consed representations binary decision diagrams also depend on store a repeated substructure once rather than rebuilding it [14].

A dependent sequence of activities poses a structurally similar problem under a different algebra. The critical path method computes the longest path through a project network and the float every other activity carries, a computation whose float calculus dates to the late 1950s [15].

Flow along capacitated edges is the fourth reading, and in its deterministic form it is the oldest of the four, dating to the max-flow-min-cut results of the 1950s [16]. What deterministic flow analysis does not address is what happens once the capacities themselves are uncertain.

0.2 Uncertainty in Network Analysis

A component's failure probability, an edge's capacity, and an activity's duration are rarely known exactly. Each is typically estimated from sparse failure data, elicited from expert judgement, or quoted as a manufacturer's tolerance, and representing that further layer of not-knowing is itself a choice with several competing answers.

Fuzzy set theory represents such a quantity as a possibility distribution rather than a probability distribution, assigning each possible value a graded degree of membership rather than a frequency [17]. Dempster-Shafer evidence theory assigns belief mass to sets of possible values directly, combining evidence from independent sources without requiring a single probability assignment over individual outcomes [18]. Imprecise probability represents the same not-knowing as a bounded set of probability distributions rather than one distribution or one possibility function: an interval bounds an unknown probability between two fixed values, and a probability box bounds an entire unknown distribution between two fixed ones [19, 20].

Reliability computation's exact methods, from reliability block diagrams through junction trees, consume a single point-valued probability throughout; the imprecise-probability treatment above is the direct answer to that limitation.

Flow under uncertainty takes forms that do not coincide with each other. Robust optimisation treats an uncertain capacity adversarially rather than probabilistically: a capacity is drawn from an uncertainty set rather than assigned a distribution, and the network is evaluated against its worst case within that set. The approach has its own complexity landscape, the robust maximum-flow problem is solvable in polynomial time, while the corresponding robust minimum-cut problem is NP-hard [21]. No probability measure enters this formulation at any point. The stochastic-flow network reliability problem, founded by Doulliez and Jamoulle, holds capacities as point-valued random variables instead and asks for the probability that the network can deliver a stated demand: each arc's capacity is a discrete random variable, and network reliability comes from decom-

posing the space of feasible capacity configurations [22]. The method family that follows, state-space decomposition, boundary-point enumeration, and sum-of-disjoint-products techniques for combining them, mirrors path and cut-set reliability computation closely enough that the same formalism travels into project scheduling directly: an activity's duration and cost stand in for an arc's capacity states, precisely the modelling move behind the exact multi-state project reliability method discussed in Section 0.3 [23].

Duration uncertainty in scheduling followed a similar split. PERT replaced each fixed duration with a three-point estimate and a distributional assumption almost as soon as the critical path method itself appeared [24]. An interval rather than a distribution over each duration is harder still: deciding whether an activity is possibly critical is NP-complete in general [25].

Across reliability, flow, and schedule, the same handful of representations recur, point-valued distributions, robust or adversarial bounds, and imprecise probability, but no single representation is used consistently across all three; each domain settled on its own answer largely independently of the other two.

0.3 Multi-Analysis Methods for Complex Infrastructure

Reasoning about a complex infrastructure system rarely stops at one question, and running more than one kind of analysis against a single shared network representation is an established pattern in engineering software, not unique to any one discipline. `pandapower`, an open-source power-system analysis tool, runs power flow, optimal power flow, state estimation, topological graph search, and short-circuit calculation against one explicit graph of buses, lines and transformers, a diversity of analysis paradigm matched by commercial power-system suites such as DIgSILENT PowerFactory and PSS/E [26]. The ArcGIS Utility Network models power, gas, and water networks as an explicit graph of features connected through topology, and runs eight distinct trace types, connectivity, isolation, subnetwork discovery, loop detection among them, against that one shared topology [27]. Neither tool's analyses are this thesis's reachability reliability, capacitated flow, or activity scheduling, and neither propagates interval or probability-box uncertainty.

Whether an existing tool or method combines reachability reliability, capacitated flow, and activity scheduling specifically, on one network representation, is a narrower question. The search for it turned up three distinct partial answers among tools and methods, plus one comparable piece of doctoral research, and none of the four meets it fully.

Commercial reliability software has, in practice, already paired reliability with flow. Several of the reliability block diagram and fault tree suites still in active use, ReliaSoft's BlockSim, Isograph's Availability Workbench, and ITEM Software's ITEM ToolKit

among them, bundle a throughput or process-flow module alongside their core reliability engine, modelling multi-flow-type capacity, bottlenecks and resource allocation on the same diagrams their reliability analyses use [28–30]. None of their public documentation describes a project-scheduling or critical-path module, and all three represent component life and failure with classical distributions, fitted from data or specified by the analyst, rather than intervals or probability boxes.

Open-source infrastructure simulation platforms have, from the opposite direction, paired flow with something schedule-shaped. InfraRisk integrates power-flow, water-hydraulic and traffic-flow simulators for interdependent power, water and road networks under its own banner of resilience analysis, and its recovery module sequences repair-crew actions with real start and end times computed from travel time and repair duration, either by heuristic prioritisation or by model-predictive-control optimisation [31]. That sequencing is a logistics schedule rather than a precedence-network critical-path computation, since it optimises the order and timing of repair actions against a performance-loss metric rather than the float of a fixed activity network. Its reliability side is likewise not a source-to-node reachability computation. The hazard module derives point component-failure probabilities from exposure and intensity to drive Monte-Carlo-style disaster scenarios, and the platform reports its results as simulated performance loss rather than an exact reliability figure.

A third direction pairs reliability with schedule directly, inside project-network reliability itself. Modelling a project's activities as the arcs of a multi-state network and asking for the probability that the project completes within both a time and a budget constraint is an established line of work. Huang, Huang and Lin's decomposition method computes this exact project reliability, splitting the space of feasible completion-time and budget combinations into subsets via a project's upper and lower boundary points and evaluating each by inclusion-exclusion, an approach the authors validate against full enumeration on a real thirteen-activity manufacturing project [23]. The method returns a project reliability of 0.906 for one tested time-budget pair on that project and traces how the figure moves as the constraints are relaxed. It has no notion of a physical arc capacity carrying a transportable flow, activity duration and cost standing in its place instead, and every input is a discrete state with a fixed probability rather than an interval or a probability box.

Doctoral research comes closer than any commercial or open-source tool to combining two of the three. Tong's Georgia Tech dissertation develops new methods for infrastructure flow networks and investigates their reliability in terms of both connectivity and flow capacity together, using a probability propagation method published the same year and belonging to the same message-passing lineage surveyed in Section 0.1 [32]. Schedule and critical-path analysis are absent from it, and the probability treatment throughout is point-valued rather than interval- or probability-box-based.

No candidate combines all three analyses on one network, and the four above do not even pair the same two twice: reliability and flow are paired in commercial software with no schedule dimension, flow and a form of scheduling are paired in open-source simulation platforms with no formal reliability computation, reliability and schedule are paired in project-network theory with no true flow network, and reliability and flow are paired again, this time exactly, in Tong's dissertation, still without schedule. Every one of the four, without exception, represents its uncertain inputs precisely or adversarially, never as an interval or a probability box, and neither `pandapower` nor the ArcGIS Utility Network above changes that picture: both run several kinds of analysis on one network, but neither runs this thesis's specific three, and neither propagates imprecision either.

0.4 Availability of Engineering Analysis Software

In engineering analysis, three patterns of software availability recur, browser-delivered simulation-as-a-service, licensed desktop installation, and web-based GIS, and none of them has been adopted by the no-code and low-code business-software movement.

The no-code and low-code software category defines itself around business application development throughout its own literature. A systematic review of 383 definitions across 306 publications from 2014 to 2025 characterises no-code platforms as visual, drag-and-drop environments that let a non-programmer build software without writing code, and low-code platforms as the same idea with a professional-developer audience layered on top [33]. Mapping the identified platforms onto ten functional system classes, the same review names enterprise-resource-planning extensions, customer-relationship platforms, multi-experience development platforms, business-process automation and management suites, robotic process automation, document management systems, and integration-platform hubs as the categories the low-code and no-code movement actually occupies, illustrated with named commercial examples such as Mendix, Salesforce, UiPath, Bizagi and Zapier. Engineering or scientific analysis, reliability, flow, structural or otherwise, does not appear among them.

In engineering analysis, browser-delivered simulation-as-a-service is one established alternative. SimScale, launched in 2013, runs computational fluid dynamics, finite element, thermal and electromagnetic analysis entirely in the browser with no local installation [34], though its computation runs in the vendor's cloud rather than on the user's own machine. The licensed desktop GUI reliability suite is a second, older pattern: the same commercial software named in Section 0.3 solves the no-programming half of the delivery problem as a local install rather than a browser application, the reverse trade-off from SimScale's.

Some web-based GIS platforms exist specifically for hazard and infrastructure risk assessment. A PRISMA-based systematic review of this literature, screening 1,775 ar-

ticles and admitting 65, organises the field around themes including visualisation and user-interface design and decision-support systems [35]. Repetto et al.'s platform is a concrete member of it. It delivers wind-monitoring, forecasting and statistical risk mapping for ports, motorways, rail lines and industrial plants exposed to wind. Its interface is web-based GIS built on free and open geospatial software, run by port authorities and operators who never touch the underlying numerical wind and wave models themselves [36]. Most platforms in this category focus on monitoring and forecasting rather than a structural reliability or component-failure computation.

Local-first software keeps the usability of the modern web while storing and computing its data on the user's own machine rather than a remote server [37], an architecture that matters most in exactly the industries whose proprietary system models cannot leave the premises.

SimScale is browser-based but not local; the desktop suites are local but not browser-based; the web-GIS platforms are browser-based but run on a remote server. No tool found in this search combines browser delivery, no programming, and local computation and data all at once.

0.5 Julia as a Language for Network Modelling and Analysis

MATLAB has offered built-in graph and network functions since the mid-2010s: shortest-path, maximum-flow, centrality, connected-component and spanning-tree computations all operate on its native `graph` and `digraph` classes [38]. R's equivalent general-purpose graph package is igraph, used for shortest paths, centrality, community detection and flow computation over one shared graph object [39], and R additionally carries a package purpose-built for structural reliability specifically, computing system and survival signatures over a network's topology [40], the kind of dedicated reliability tooling neither MATLAB's built-in functions nor Python's ecosystem, below, provide natively. Rust's graph ecosystem is newer and smaller, centred on `petgraph`, a general-purpose graph data-structure library with the same shortest-path, spanning-tree and traversal algorithms as the languages above [41].

Python's equivalent general-purpose graph package is NetworkX, whose own canonical paper presents it as covering simple graphs, directed graphs, and graphs with parallel edges, over a single native numeric type [42]. No single Python package purpose-built for network reliability computation, in the reliability block diagram, path-set, or decision-diagram sense this literature uses the term, was found in this search, the way R's `ReliabilityTheory` package is; the closest candidate, `pgmpy`, is a general Bayesian-network and probabilistic-graphical-model toolkit with belief propagation and structure

learning built in, useful machinery for the same class of problem but not a package built to solve it directly. This is a gap in packaged tooling specifically, not in published research: network reliability analysis is itself an actively reviewed field with applications across communications, transportation, and electrical and gas distribution [43], and that research draws on whichever language a given group already works in without converging on one shared package the way NetworkX did for general graph algorithms.

Julia's own graph ecosystem, centred on `Graphs.jl` and its satellite packages for weighted graphs, property graphs and bindings to established external libraries, occupies the equivalent general-purpose position to NetworkX, igraph and `petgraph`, though no single paper describes that ecosystem the way Hagberg, Schult and Swart's paper describes NetworkX. None of the packages named above, in any of the five languages, carries a native notion of an interval- or probability-box-valued edge or node, which is not a limitation any of them sets out to address, since none is built for propagating uncertain quantities across a network in the first place.

Multiple dispatch, choosing which method of a generic function to run from the run-time types of every argument rather than just the first, is presented in the canonical Julia paper as the mechanism that lets one generic function definition serve arbitrarily many numeric and non-numeric types without its author anticipating any of them in advance [44]. The same paper grounds Julia's central design claim, that a dynamic language need not trade its ease of use for the performance a statically compiled one gets, in the same type system and dispatch mechanism. This is the mechanism this framework's own propagation step leans on to specialise correctly whether the value flowing through it is a float, an interval, or a probability box, without the author of the propagation step anticipating any of the three in advance.

0.6 Summary

Reliability, flow, and schedule analysis grew as three separate literatures, each with its own exact methods and its own way of representing what it does not know precisely about its inputs. Running several kinds of analysis against one shared network object is not itself a new idea in engineering software, but no tool, method, or doctoral thesis found in this search combines reachability reliability, capacitated flow, and activity scheduling on one network representation, with any of them propagating imprecision natively across the combination. No engineering-analysis tool found in this search delivers that combination through an interface that asks its user for files and clicks rather than code, either. This thesis's own framework is built to close that gap. It computes all three analyses from one network model, propagates interval uncertainty natively through each and probability-box uncertainty through reliability specifically, and reaches its user through a local, no-code interface rather than a script.

Bibliography

[1] R. Billinton and R. N. Allan, "Network modelling and evaluation of simple systems," in *Reliability Evaluation of Engineering Systems: Concepts and Techniques*, Boston, MA: Springer US, 1992.

[2] L. Xing and S. V. Amari, "Fault tree analysis," in *Handbook of Performability Engineering*, London: Springer London, 2008.

[3] W. Liu and J. Li, "An improved cut-based recursive decomposition algorithm for reliability analysis of networks," *Earthquake Engineering and Engineering Vibration*, vol. 11, no. 1, pp. 1–10, 2012.

[4] Y. Kim and W.-H. Kang, "Network reliability analysis of complex systems using a non-simulation-based method," *Reliability Engineering & System Safety*, vol. 110, pp. 80–88, 2013.

[5] G. S. Fishman, "A comparison of four Monte Carlo methods for estimating the probability of s–t connectedness," *IEEE Transactions on Reliability*, vol. 35, no. 2, pp. 145–155, 1986.

[6] R. E. Bryant, "Graph-based algorithms for Boolean function manipulation," *IEEE Transactions on Computers*, vol. C-35, no. 8, pp. 677–691, 1986. DOI: 10.1109/TC.1986.1676819

[7] A. Rauzy, "New algorithms for fault trees analysis," *Reliability Engineering & System Safety*, vol. 40, no. 3, pp. 203–211, 1993.

[8] J. Pearl, "Probabilistic reasoning in intelligent systems: Networks of plausible inference," 1988.

[9] S. L. Lauritzen and D. J. Spiegelhalter, "Local computations with probabilities on graphical structures and their application to expert systems," *Journal of the Royal Statistical Society, Series B*, vol. 50, no. 2, pp. 157–224, 1988.

[10] A. Satyanarayana and R. K. Wood, "A linear-time algorithm for computing K-terminal reliability in series-parallel networks," *SIAM Journal on Computing*, vol. 14, no. 4, pp. 818–832, 1985. DOI: 10.1137/0214057

[11] A. Satyanarayana and M. K. Chang, "Network reliability and the factoring theorem," *Networks*, vol. 13, no. 1, pp. 107–120, 1983. DOI: 10.1002/net.3230130107

[12] R. K. Wood, "A factoring algorithm using polygon-to-chain reductions for computing K-terminal network reliability," *Networks*, vol. 15, no. 2, pp. 173–190, 1985. DOI: 10.1002/net.3230150204

[13] A. K. Goharshady and F. Mohammadi, "An efficient algorithm for computing network reliability in small treewidth," *Reliability Engineering & System Safety*, vol. 193, p. 106 665, 2020. DOI: 10.1016/j.ress.2019.106665

[14] J.-C. Filliâtre and S. Conchon, "Type-safe modular hash-consing," in *Proceedings of the ACM SIGPLAN Workshop on ML*, 2006, pp. 12–19. DOI: 10.1145/1159876.1159880

[15] J. E. Kelley and M. R. Walker, "Critical-path planning and scheduling," in *Proceedings of the Eastern Joint IRE-AIEE-ACM Computer Conference*, Boston, MA, 1959, pp. 160–173. DOI: 10.1145/1460299.1460318

[16] L. R. Ford and D. R. Fulkerson, "Maximal flow through a network," *Canadian Journal of Mathematics*, vol. 8, pp. 399–404, 1956. DOI: 10.4153/CJM-1956-045-5

[17] L. A. Zadeh, "Fuzzy sets," *Information and Control*, vol. 8, no. 3, pp. 338–353, 1965.

[18] G. Shafer, *A Mathematical Theory of Evidence*. Princeton, NJ: Princeton University Press, 1976.

[19] S. Ferson, V. Kreinovich, L. Ginzburg, D. S. Myers, and K. Sentz, "Constructing probability boxes and Dempster–Shafer structures," Sandia National Laboratories, Albuquerque, NM, Tech. Rep. SAND2002-4015, 2003.

[20] R. C. Williamson and T. Downs, "Probabilistic arithmetic I: Numerical methods for calculating convolutions and dependency bounds," *International Journal of Approximate Reasoning*, vol. 4, no. 2, pp. 89–158, 1990.

[21] D. Bertsimas, E. Nasrabadi, and S. Stiller, "Robust and adaptive network flows," *Operations Research*, vol. 61, no. 5, pp. 1218–1242, 2013. DOI: 10.1287/opre.2013.1200

[22] P. Doulliez and E. Jamoulle, "Transportation networks with random arc capacities," *Revue française d'automatique, informatique, recherche opérationnelle. Recherche opérationnelle*, vol. 6, no. V3, pp. 45–59, 1972. [Online]. Available: https://www.numdam.org/item/RO_1972__6_3_45_0/

[23] D.-H. Huang, C.-F. Huang, and Y.-K. Lin, "Exact project reliability for a multi-state project network subject to time and budget constraints," *Reliability Engineering & System Safety*, vol. 195, p. 106 744, 2020. DOI: 10.1016/j.ress.2019.106744

[24] D. G. Malcolm, J. H. Roseboom, C. E. Clark, and W. Fazar, "Application of a technique for research and development program evaluation," *Operations Research*, vol. 7, no. 5, pp. 646–669, 1959. DOI: 10.1287/opre.7.5.646

[25] S. Chanas and P. Zieliński, "The computational complexity of the criticality problems in a network with interval activity times," *European Journal of Operational Research*, vol. 136, no. 3, pp. 541–550, 2002. DOI: 10.1016/S0377-2217(01)00048-0

[26] L. Thurner et al., "Pandapower—an open-source python tool for convenient modeling, analysis, and optimization of electric power systems," *IEEE Transactions on Power Systems*, vol. 33, no. 6, pp. 6510–6521, 2018. DOI: 10.1109/TPWRS.2018.2829021

[27] Esri, *ArcGIS utility network: Trace types*, https://doc.esri.com/en/arcgis-pro/latest/help/data/utility-network/utility-network-trace-types.html, 2024.

[28] Hottinger Brüel & Kjær (ReliaSoft). "BlockSim: System reliability, availability, and maintainability analysis."

[29] Isograph Ltd. "Availability Workbench."

[30] ITEM Software. "ITEM ToolKit: Reliability analysis and safety software tools."

[31] S. Balakrishnan and B. Cassottana, "InfraRisk: An open-source simulation platform for resilience analysis in interconnected power–water–transport networks," *Sustainable Cities and Society*, vol. 83, p. 103 963, 2022. DOI: 10.1016/j.scs.2022.103963

[32] Y. Tong, "New approaches for modeling and reliability assessment of infrastructure flow networks," Ph.D. dissertation, Georgia Institute of Technology, 2021.

[33] A. Abendroth, "The democratization of software engineering: Evolution, definition, and the future of low-code and no-code platforms," *Management Review Quarterly*, vol. 67, 2026. DOI: 10.1007/s11301-026-00603-2

[34] SimScale GmbH, *SimScale: AI-native engineering simulation software in the cloud*, https://www.simscale.com/, 2026.

[35] M. Daud, F. M. Ugliotti, and A. Osello, "Comprehensive analysis of the use of web-gis for natural hazard management: A systematic review," *Sustainability*, vol. 16, no. 10, p. 4238, 2024. DOI: 10.3390/su16104238

[36] M. P. Repetto, M. Burlando, G. Solari, P. De Gaetano, M. Pizzo, and M. Tizzi, "A web-based GIS platform for the safe management and risk assessment of complex structural and infrastructural systems exposed to wind," *Advances in Engineering Software*, vol. 117, pp. 29–45, 2018. DOI: 10.1016/j.advengsoft.2017.03.002

[37] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, "Local-first software: You own your data, in spite of the cloud," in *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, ser. Onward! 2019, ACM, 2019, pp. 154–178. DOI: 10.1145/3359591.3359737

[38] The MathWorks, Inc., *Graph and network algorithms*, https://www.mathworks.com/help/matlab/graph-and-network-algorithms.html, 2026.

[39] G. Csárdi and T. Nepusz, "The igraph software package for complex network research," *InterJournal, Complex Systems*, p. 1695, 2006.

[40] L. J. M. Aslett, *ReliabilityTheory: Tools for structural reliability analysis*, https://cran.r-project.org/package=ReliabilityTheory, 2024.

[41] petgraph contributors, *Petgraph: Graph data structure library for Rust*, https://github.com/petgraph/petgraph, 2026.

[42] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in *Proceedings of the 7th Python in Science Conference (SciPy2008)*, G. Varoquaux, T. Vaught, and J. Millman, Eds., Pasadena, CA, 2008, pp. 11–15. DOI: 10.25080/TCWV9851

[43] V. Gaur, O. P. Yadav, G. Soni, and A. P. S. Rathore, "A literature review on network reliability analysis and its engineering applications," *Proceedings of the Institution of Mechanical Engineers, Part O: Journal of Risk and Reliability*, vol. 235, no. 2, pp. 167–181, 2021.

[44] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah, "Julia: A fresh approach to numerical computing," *SIAM Review*, vol. 59, no. 1, pp. 65–98, 2017. DOI: 10.1137/141000671